Rapport – Étape 02 : API & base de données

Programmation Web avancée [420-WA6-AG] — Automne 2026

1. Justification du modèle de données

Normalisation. Le schéma est en 3FN : chaque fait n'est stocké qu'une fois. Les relations plusieurs-à-plusieurs (panier ↔ ouvrage, commande ↔ ouvrage, liste ↔ ouvrage) passent par des tables d'association (`panier_items`, `commande_items`, `liste_items`). Seule duplication volontaire : `commande_items.prix_unitaire`, qui fige le prix payé (un changement de prix ultérieur ne doit pas modifier l'historique). Les indicateurs dérivés (note moyenne, popularité) sont **calculés** à la lecture plutôt que stockés, ce qui évite toute incohérence.

Relations et suppression. Les clés étrangères utilisent `CASCADE` pour les données purement dépendantes (items, avis, paniers) et `RESTRICT` pour l'historique (un ouvrage déjà vendu ou un client ayant des commandes ne peut pas être supprimé). Les utilisateurs sont désactivés (`actif = FALSE`) au lieu d'être supprimés.

Contraintes d'intégrité au niveau SGBD. `CHECK (stock >= 0)`, `CHECK (prix >= 0)`, `CHECK (note BETWEEN 1 AND 5)`, `CHECK (quantite > 0)`, `UNIQUE (client_id, ouvrage_id)` sur `avis`, `UNIQUE (panier_id, ouvrage_id)`, `UNIQUE (code_partage)`, ainsi que `quantite_achetee <= quantite_souhaitee` dans les listes. Ainsi, même un bug applicatif ne peut pas produire un stock négatif ou un double avis.

Index. Hors PK et UNIQUE : `ouvrages(categorie_id)`, `ouvrages(stock)` (filtre `stock > 0`), `ouvrages(titre)` et `(auteur)` (recherche), `commande_items(ouvrage_id)` (popularité et vérification d'achat), `commandes(client_id)` et `(statut)`, `panier(actif, updated_at)` (script d'expiration), `commentaires(ouvrage_id, valide)` (affichage des commentaires validés) et `(valide, rejete)` (file de modération).

2. Implémentation des règles métier

Règle	Où / comment
Seuls les ouvrages avec stock > 0 sont visibles	`ouvragesService.list` ajoute `WHERE stock > 0` ; le détail d'un ouvrage épuisé renvoie 404 au public (le personnel peut le consulter).
Décrément transactionnel du stock	`commandesService.creerCommande` s'exécute dans une transaction : `SELECT … FOR UPDATE` sur chaque ouvrage (ordre d'id croissant pour éviter les deadlocks), vérification du stock, `UPDATE stock = stock - ?`, création de la commande, des lignes et du paiement. Si une ligne manque de stock → erreur 409 et **rollback complet** (testé : aucun stock modifié). Le `CHECK (stock >= 0)` sert de filet de sécurité.
Avis réservé aux acheteurs	`avisService.ajouterAvis` vérifie qu'une commande du client, au statut `payee` ou `expediee`, contient l'ouvrage (403 sinon). Une commande annulée n'ouvre donc pas ce droit. La contrainte `UNIQUE` garantit un seul avis par client/ouvrage (409).
Commentaires modérés	Insertion avec `valide = FALSE`. Seul un éditeur (middleware de rôle) appelle `PUT /api/commentaires/:id/valider` avec `decision = valider` ou `rejeter` ; la date et le modérateur sont enregistrés. Seuls les commentaires validés sont renvoyés avec le détail d'un ouvrage.
Panier expiré	`npm run clean:paniers` supprime les paniers actifs inactifs depuis N jours (`PANIER_EXPIRATION_JOURS`, 7 par défaut) ; exécutable via cron. Un panier est archivé (`actif = FALSE`) à la validation de la commande.
Liste de cadeaux	Code de partage aléatoire (6 octets, `crypto`). L'achat direct réutilise la même transaction de commande et incrémente `quantite_achetee` sous verrou (`FOR UPDATE`), sans dépasser la quantité souhaitée.

Authentification et sécurité. Mots de passe hachés avec bcrypt (`bcryptjs`, 10 tours) ; JWT signé (expiration 2 h) ; le middleware relit l'utilisateur en base à chaque requête (rôle à jour, compte désactivé refusé immédiatement). Le rôle est toujours `client` à l'inscription. Les routes sensibles sont protégées par `authenticate` + `requireRoles(...)` (l'administrateur passe toujours). Toutes les entrées sont validées avec Joi (middleware), les requêtes SQL sont paramétrées (pas d'injection), les erreurs sont traitées par un gestionnaire centralisé, `helmet` est activé, et aucune donnée bancaire n'est stockée.

3. Limitations connues et améliorations possibles

- **Paiement simulé** : la commande est créée directement au statut `payee` avec une URL fictive. Une vraie intégration (Stripe, PayPal) devrait créer la commande `en_cours` puis la confirmer via un webhook.
- **Un seul panier actif par client** est garanti par le code, pas par la base ; une colonne générée avec index unique pourrait le renforcer.
- **Recherche** par `LIKE` : un index `FULLTEXT` serait plus performant sur un gros catalogue.
- **Authentification** : pas de refresh token, de révocation ni de limitation du nombre de tentatives de connexion (`express-rate-limit`).
- **Prix du panier** : la commande utilise le prix courant de l'ouvrage ; le panier n'affiche que le prix d'ajout à titre indicatif.
- **Tests automatisés** (Jest + Supertest) et migrations versionnées (Knex/Flyway) à ajouter ; documentation OpenAPI/Swagger.